

# Computational Mechanics

Xavier Poncini  
Simplex

Adam Shai  
Simplex

Paul Riechers  
Simplex

## 1 Prerequisites

Familiarity with the goals and basic methodology of mechanistic interpretability, including the notions of features and circuits; the linear representation hypothesis; linear probes as a method for reading off internal representations; and a basic understanding of the transformer architecture.

Students should be comfortable with the idea that one can train a linear map from model activations to some target structure and evaluate its quality (e.g. via MSE or  $R^2$ ).

**Mathematical background.** The module is mathematically self-contained: all formal definitions (HMMs, GHMMs, belief states, MSPs) are introduced from scratch. However, students will engage with the material more fluently if they are comfortable with the following:

- **Linear algebra:** row-stochastic matrices, rank and invertibility, null spaces (kernels).
- **Probability:** conditional probability, Bayes' rule, probability distributions over finite sets, the probability simplex.
- **Basic machine learning:** next-token prediction, loss functions (cross-entropy), the concept of model activations.

### What you'll learn

- Construct their own hidden Markov models.
- Explain the motivation for introducing generalised hidden Markov models (GHMMs).
- Explain why the belief state is a useful object for making predictions about GHMM data.
- Compute the belief state of a GHMM.
- Explain the difference between the belief state of a HMM and the belief state of a GHMM (that is itself not an HMM).

- Explain the relationship between belief states and next-token observation probabilities for GHMMs.
- Compute the mixed state presentation (MSP) for GHMMs.
- Explain why generating data is easier than predicting data.
- Explain the evidence for the claim: transformers represent belief state geometry in their residual stream.
- Develop hypotheses for where transformers represent: belief states, observation probabilities, log observation probabilities.

## 2 Content

### 2.1 Fast-track

Read [Transformers represent belief state geometry in their residual stream](#). Focus on being able to answer the following questions:

- What is a hidden Markov model (HMM)?
- What is a belief state?
- What is the mixed state presentation (MSP)?
- About the map learnt from activations to belief states:
  - What is the source?
  - What is the target?
  - How is the quality of the map evaluated?

### 2.2 Main content

#### 1. [Lecture slides](#) Overview and scope

- (a) What is computational mechanics?
- (b) Computational mechanics & AI safety
- (c) Where is the research program at?
- (d) What will we cover today?

#### 2. Computational mechanics foundations

- (a) [Lecture Slides](#) & [Exercise Sheet](#) Hidden Markov Models (HMMs)
  - i. What is an HMM?
  - ii. Notions of minimality & uniqueness

- iii. What is a generalised HMM (GHMM)? (see e.g. [Neural networks leverage nominally quantum and post-quantum representations](#))
- (b) [[Lecture slides](#) & [Exercise Sheet](#)] **Prediction**
  - i. Belief states
  - ii. Next-token (probability) vectors
  - iii. Mixed-state-presentation (MSP) (see e.g. [Mixed States of Hidden Markov Processes and Their Presentations: What and how to calculate](#))

### 3. Applications

- (a) [[Tutorial Sheet](#)] **Designing processes worth studying**
  - i. Heuristic ensuring beliefs are meaningfully richer than obs probs
  - ii. Non-ergodic
  - iii. Factors (see e.g. [Transformers learn factored representations](#))
- (b) [[Reading guide](#)] **Identifying belief geometry in transformers**
  - i. For the case of HMMs – [Read: [Transformers represent belief state geometry in their residual stream](#)].
  - ii. [extension] For the case of GHMMs – [Read: [Neural networks leverage nominally quantum and post-quantum representations](#)]
- (c) [[Tutorial Sheet](#)] **How do transformers do it?**
  - i. How do transformers construct representations of belief states? Partial answer in narrow setting – [Read: [Constrained belief updates explain geometric structures in transformer representations](#)]
  - ii. How do they decode from belief states to log obs probs then obs probs? This is an open problem!

## 3 Hidden Markov models

*Below are exercises accompanying the [lecture](#) on hidden Markov models (HMMs). In this session, you will implement a GHMM from scratch and use it to compute the probability of observed token sequences – the foundational machinery you will build on in later blocks. Spend ~30 minutes on these exercises; start with the core set, and progress to the extension if you have time.*

### 3.1 Core exercises

**Exercise 3.1.** Using your programming language of choice, implement a function that takes in:

- (a) The transition matrices of a GHMM (a three-tensor)
- (b) The initial vector

(c) A sequence of tokens

and outputs the probability of observing that particular sequence of tokens.

**Exercise 3.2.** Test your function on the random-random-XOR (RRXOR) process when initialised in the state  $\eta^{(0)} = [1\ 0\ 0\ 0\ 0]$ . Make sure you assign zero probability to XOR violations.

**Exercise 3.3.** Convince yourself that that the definition of a GHMM allows one to interpret  $\eta^{(0)}T^{(w_1)}T^{(w_2)}\dots T^{(w_n)}\phi$  as the probability of emitting the sequence  $w_1w_2\dots w_n$ .

**Exercise 3.4.** Derive the transition matrices for the zero-random-random process.

## 3.2 Extension exercises

**Exercise 3.5.** The RRXOR process can be viewed as a particular instance of a more general process called *random-random-Mod p* (RRModp) where  $p = 2, 3, \dots$ , and  $\text{RRXOR} = \text{RRMod}2$ .

- (a) What are the transition matrices for the RRMod3 process?
- (b) What are the transition matrices for the RRModp process?
- (c) Do you notice anything about the matrices sitting in the blocks?

**Exercise 3.6.** The RRModp process can be similarly viewed as a particular instance of a more general process that computes the product of two *group* elements (for RRModp the cyclic group is the relevant one). Identify another group whose multiplication can be expressed as an HMM.

## 4 Prediction

*Below are exercises accompanying the [lecture](#) on prediction. Building on your GHMM implementation from [Block 2](#), you will now compute belief states and next-token vectors, and use them to probe the predictive structure of canonical processes (Z1R, Mess3, RRXOR). Spend ~30 minutes on these exercises; start with the core set, and progress to the extension if you have time.*

## 4.1 Core exercises

**Exercise 4.1.** Using your programming language of choice, implement a function that takes in:

- (a) The transition matrices of a GHMM (a three-tensor)
- (b) The initial vector
- (c) A sequence of tokens

and outputs the corresponding: belief state and next-token (prediction) vector.

**Exercise 4.2.** Test your function on the zero-one-random (Z1R) process when initialised in the state  $\eta^{(\emptyset)} = [1/3 \ 1/3 \ 1/3]$ , and compare to the results of the worked example on [Slide 12](#).

**Exercise 4.3.** Visualise belief state geometry and the next-token geometry for the [Mess3 process](#) initialised in the uniform distribution. Does it look like the theoretical prediction in Fig. 1 of [Shai et al](#)?

**Exercise 4.4.** Derive the belief state update rule on [Slide 5](#).

**Exercise 4.5.** Show that the matrix elements of the matrix taking beliefs to next-token probabilities are given by expression stated on [Slide 7](#).

## 4.2 Extension exercises

**Exercise 4.6.** Evaluate whether the linear map from beliefs to observation probabilities is invertible for:

- (a) RRXOR
- (b) Mess3

**Exercise 4.7.** Complete the MSP of the Z1R process (explored in the lecture) by annotating the tokens with the correct emission probabilities.

**Exercise 4.8.** Compute the MSP for the zero-random-random process.

**Exercise 4.9.** For an element  $a \in \mathcal{X}^*$ , the corresponding *causal state equivalence class* is given by

$$[a]_C := \{x \in \mathcal{X}^* \mid \Pr(y \mid a) = \Pr(y \mid x), \forall y \in \mathcal{X}^*\}.$$

Likewise, the corresponding *belief state equivalence class* is given by

$$[a]_B := \{x \in \mathcal{X}^* \mid \eta^{(a)} = \eta^{(x)}\}.$$

How are these sets related to each other?

## 5 Identifying belief geometry in transformers

*Up to this point, we have focused on conceptual aspects of computational mechanics; now we will consider whether this framework applies to neural networks (e.g., transformers & RNNs). In this session, you will read the key sections of a few select papers, then discuss the findings in small groups.*

1. **Readings (45 minutes).** Follow the guide outlined in the Readings Section.
2. **Discussion (15 minutes).** Form groups of 3-4 based on how much you read. Discuss aspects of the readings you find interesting/confusing. See the Discussion Section for prompts you may wish to consider.

### 5.1 Readings

This reading session consists of a main reading and an extension reading. Focus on the main reading, and proceed to the extension reading if you have time. If you can answer the ‘Evaluation’ question it is likely you have understood a key component of the reading. Feel free to skim the overview of the extension reading if you don’t have time to read it.

#### 5.1.1 [Main] [Transformers represent belief state geometry in their residual stream](#)

Reading suggestion:

- **Skim.** Sections 1, 2.1 & 2.2.
- **Read.** Sections 2.3, 3–5.
- **Evaluation.** Explain Figure 6 D.

**Overview.** This was the first paper to present evidence that transformers trained on HMM data admit linear(!) representations of the belief state in their residual stream. Evidence for this claim is presented for two processes Mess3 & random-random XOR

(RRXOR). For each process linear regression is used to identify an affine map from residual stream activations to the corresponding belief state, which is then evaluated by computing the mean-squared error (MSE). MSE is evaluated across model checkpoints and is shown to decrease through training. A shuffle-control is used to illustrate that the low MSE from activations to beliefs is not simply explained by projecting from a high-dimensional space (the activations) to a low-dimensional space (beliefs).

### 5.1.2 [Extension] Neural networks leverage nominally quantum and post-quantum representations

**Reading suggestion:**

- **Skim.** Sections 1–3
- **Read.** Sections 4–8.
- **Evaluation.** Explain Figure 3 B. How are light-blue (orange) plots related to the blue (orange) plots?

**Overview.** This paper presents evidence that the [Shai et al.](#) result extends to generalised hidden Markov Models (GHMMs) – a strictly more expressive class of processes than HMMs, allowing for quantum and post-quantum representations. To illustrate, there exists stochastic processes that admit realisation in terms of a GHMM with finite-dimensional transition matrices, while the corresponding minimal HMM requires exponentially many hidden states.

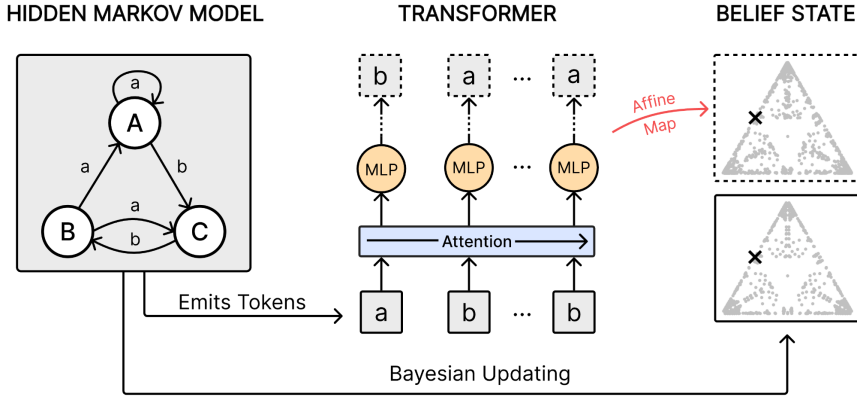
Using techniques similar to [Shai et al.](#), it is shown that neural networks (transformers, LSTMs, RNNs, GRUs) trained on stochastic processes admitting distinct GHMM and HMM realisations learn belief state representations corresponding to the GHMM. This demonstrates the ability of neural networks to flexibly select the model class underlying their representations to capture exponential savings.

## 5.2 Discussion

Here are some discussion prompts to consider:

- What is the most convincing evidence that transformers represent belief state geometry?
- What is the least convincing?
- What additional experiments would you run to gain more evidence?
- Are there alternative hypotheses that explain these results?
- Can you explain each of element of the following diagram:

*(Feel free to ignore these if you/your group is excited to explore other aspects of the readings.)*



## 6 Designing processes

In this session, you will design your own hidden Markov model (HMM) and reuse code from previous exercises to explore its structure. Not all HMMs are interesting — use the [Desiderata Section](#) to guide your design toward processes with meaningful properties. You will then present your process to the class.

1. **Form into pairs.** Find a partner to collaborate with on the design exercise.
2. **Design your process (90 minutes).** Using the [Desiderata Section](#) as your guide, construct an HMM. Use the code from previous exercises to visualise the next-token probability vector and the belief state geometries.
3. **Present your process to the class (5 minutes).** Describe your process and demonstrate its key features, including the observation probabilities, belief state geometry, and any notable structural properties.

### 6.1 Notation

(Feel free to skip straight to the [Desiderata](#) section and refer back when necessary.)

Here, we consider data generated from a generalised hidden Markov model (GHMM), and introduce the following notation:

- $\mathcal{X}$  – a set corresponding to the dictionary ( $|\mathcal{X}| = \text{\#tokens}$ )
- $(T^{(x)})_{x \in \mathcal{X}}$  – transition matrices ( $\text{\#hidden states} \times \text{\#hidden states}$ )
- $\eta^{(0)}$  – the initial state row vector ( $1 \times \text{\#hidden states}$ )
- $\phi$  – the final state column vector ( $\text{\#hidden states} \times 1$ ).

We also introduce the following notation for token sequences, products of transition matrices and sequences of up to length  $L$

$$x_{1:\ell} := x_1 x_2 \cdots x_\ell, \quad T^{(x_{1:\ell})} := T^{(x_1)} T^{(x_2)} \cdots T^{(x_\ell)}, \quad x_{1:L} := \bigcup_{i=1}^L \mathcal{X}^i.$$



Given the observed token sequence  $x_{1:\ell} \in \mathcal{X}^\ell$ , the *belief state* can be expressed in terms of HMM data as

$$\eta^{(x_{1:\ell})} = \eta^{(\emptyset)} T^{(x_{1:\ell})} / \eta^{(\emptyset)} T^{(x_{1:\ell})} \phi.$$

Similarly the vector of next token *observation probabilities* is given by

$$p^{(x_{1:\ell})} = (\eta^{(x_{1:\ell})} T^{(x_{\ell+1})} \phi)_{x_{\ell+1} \in \mathcal{X}}.$$

Consulting the previous two expressions, we see that there exists a matrix  $A$  that takes beliefs  $\eta^{(x_{1:\ell})}$  to observation probabilities  $p^{(x_{1:\ell})}$ , i.e.

$$p^{(x_{1:\ell})} = \eta^{(x_{1:\ell})} A$$

where  $A$  is a matrix whose elements are given by

$$A_{ij} := \sum_k T_{ik}^{(j)} \phi_k.$$

We note that for GHMMs, there always exists this linear map from beliefs to observation probabilities, however, it is generically not the case that this map is invertible.

## 6.2 Desiderata

We now list a series of properties (in decreasing level of importance) that we would like our process to have.

**No affine map from observation probabilities to beliefs.** One of the central claims of comp-mech applied to neural networks is that beliefs are instrumental to predicting observation probabilities. If there exists an affine map from observation probabilities to beliefs, then, at the level of linear representations, these objects are indistinguishable and the claim of comp-mech is empty. Concretely, the desired property is that the matrix  $A$  is not invertible. A nice heuristic for this is that  $\# \text{hidden states} > \# \text{vocab}$ .

We may also want to require that there exists no linear map from observation probabilities to *approximate* beliefs. One way we might quantify this is to compute the  $R^2$  of the regression from observation probabilities to beliefs, and ensure that it satisfies:

$$R^2(\text{obs probs} \rightarrow \text{beliefs}) < R^2(\text{activations} \rightarrow \text{beliefs})$$

Intuitively, this constraint ensures that the transformer’s representation of beliefs cannot be simply explained by the model representing obs probs.

In practice, I like to select processes where  $\# \text{hidden states} > \# \text{vocab}$ , and where the beliefs occupy a  $(\# \text{hidden states} - 1)$ -dimensional region of the simplex. I also like to indicate, on the belief simplex, the directions in the kernel of the map from beliefs to observation probabilities. This allows one to see which belief states collapse to the same observation probabilities.

**Easy to visualise.** It is convenient to be able to visualise the ground truth belief states, observation probabilities and log observation probabilities without having to use dimensionality reduction techniques. As such, we have the condition:

# hidden states  $\leq 4$ .

As we are also (likely) imposing that # hidden states  $>$  # vocab, the dimensions of observation probabilities and log observation probabilities are readily handled. In practice, I like # hidden states = 3 & # vocab = 2.

**Striking belief geometry.** It is helpful to be able to quickly glance at e.g., the PCA of activations, and be able to identify patterns that correspond to some structure in the belief geometry. As such, it is nice if the belief geometry has some particular structure that distinguishes it from the observation probabilities.

**Natural properties.** While HMMs lack expressivity of other non-linear models, one can still endow them with properties shared by natural language, for example:

- **Factors.** Natural language often consists of many distinct, independent components that each influence the output distribution. For example, a character's name determines which pronouns appear; the setting (beach vs. courtroom) shapes vocabulary; the genre (horror vs. comedy) affects tone; and the tense constrains verb forms. These components combine to determine what text is likely to follow, yet operate largely independently – knowing a story is set on a beach tells you little about whether the protagonist is named Alice or Bob.
- **Non-ergodicity.** The training corpus of LLMs is a mixture of text from diverse sources (novels, code, lawsuit filings, and more), each with its own distinct statistical structure. This can naturally be characterised as a non-ergodic process that samples from a distribution over a set of component processes (which may themselves be non-ergodic). Each component process corresponds to a broad category of text: one governing how novels are generated, another how code is written, another how legal documents are structured, and so on.
- **Algorithms.** Many simple algorithms appear naturally in language. For example, answering "two days after Saturday is?" requires modular addition over days of the week, while "what is the reverse of 'cat'?" requires a reversal operation. The core logic of such algorithms can be encoded as hidden Markov models (e.g., modular addition for the days-of-the-week example), making HMMs a natural framework for studying how language models learn and represent these computations.

## 7 How do transformers do it?

*In the previous session, we explored evidence that neural networks represent belief state geometry of HMM processes. Based on your interest, we will now split into two tracks, each addressing a question:*

1. **Sensors.** *How might the transformer develop representations of the belief state?*

2. **Actuators.** *How might the transformer use belief state representations to influence downstream predictions?*

*The Sensors question is marginally better understood and will centre around reading [Piotrowski et al.](#), which partially answers this question for single-layer transformers trained on the Mess3 process. The Actuators question is relatively less explored, and is more conceptual.*

## 7.1 Sensors

**Reading suggestion (45 minutes):**

- **Skim.** Sections 1 & 2.
- **Read.** Sections 3–5.
- **Evaluation.** Explain Figure 2.

**Overview.** Given the architectural constraints of a transformer block, how might belief states be constructed out of raw token embeddings? For the case of single-layer transformers trained on the Mess3 process, [Piotrowski et al.](#) identify an algorithm implemented by the attention mechanism that transforms input sequences into *constrained beliefs* (Equ. 6), which are then transformed into the corresponding belief state by the MLP layer.

The key ansatz is that the attention block outputs constrained beliefs. This observation is then used to anticipate structure in various components relevant to the attention mechanism:

- Attention pattern (see Equ. 13)
- OV-vectors (see Equ. 14)
- Embeddings (see Equ. 15)
- Number of attention heads (see Sec. 4.4.1)

These predictions are then empirically verified across four different Mess3 parameterisations.

**Discussion (15 minutes).** Form groups of 3-4 and discuss aspects of the readings you found interesting/confusing. Here are some discussion prompts to consider:

- How is the constrained belief geometry related to the full belief geometry?
- What is the role of the MLP in transforming the constrained belief geometry into the full belief geometry?
- Do you expect the constrained belief state update formula to apply to:

- Processes beyond Mess3?
- Attention blocks with more than two heads?
- Transformers that are deeper than a single layer?

*(Feel free to ignore these if you/your group is excited to explore other aspects of the reading.)*

## 7.2 Actuators

**Background.** Suppose the transformer represents the belief state corresponding to a given input sequence at some residual stream position. Given the architectural constraints of the transformer consider the questions:

- How might the belief state be used to compute the corresponding next-token probability distribution?
- Which layer is the belief state most likely to be used by the transformer?

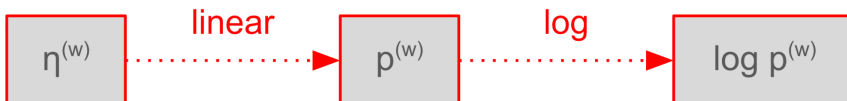
Recall that there always exists a linear map from the belief state to the next-token probabilities vector:



But given the transformer architecture, the next-token probabilities are produced by softmaxing logits (activations post-unembedding), which are related to the probabilities by

$$\text{logits}(w) = \log p^{(w)} + c_w \mathbf{1}$$

where  $c_w$  is an arbitrary  $w$ -dependent constant and  $\mathbf{1}$  is the all-ones vector of appropriate size. This observation motivates considering three types of features: *beliefs* ( $\eta^{(w)}$ ), *next-token probabilities* ( $p^{(w)}$ ) & *next-token log-probabilities* ( $\log p^{(w)}$ ); and considering how they relate to each other. Without assumptions on the HMM process, they are related as follows:



### 7.2.1 Exercises

**Exercise 7.1.** Annotate the transformer diagram with where you would expect the trained models to represent: beliefs, next-token probabilities & next-token log-probabilities (if at all) under the following assumptions:

- (a) No assumptions.
- (b) There exists a linear map from next-token probabilities to next-token log-probabilities.
- (c) There exists a linear map from next-token probabilities to beliefs.

*(Keep in mind the expressiveness of the transformer components as you pass between beliefs / next-token probabilities / log next-token probabilities.)*

**Exercise 7.2.** Design intervention experiments to gain evidence for your hypotheses.

## 8 Learn more

- Beyond toy architectures – in-context learning
  - LLMs have an amazing ability to adapt internal representations in-context – [Read: ICLR: [In-Context Learning of Representations](#)]
  - LLMs can predict HMM data in-context – [Read: [Pre-trained Large Language Models Learn Hidden Markov Models In-context](#)]
  - This suggests that internal representations of LLMs predicting GHMM data in-context, may resemble belief states.
- Processes consisting of GHMMs with richer structure [Read: [Transformers learn factored representations](#)]
- Review article of computational mechanics [Read: [Between order and chaos](#)]
- [Simplex](#) is a non-profit research organisation working on applying computational mechanics to AI safety.

